UTM online

Universidad Técnica de Manabí desde 1952

TECNOLOGÍAS DE LA INFORMACIÓN
Desarrollo de Sistemas Informáticos
4 créditos

Profesores:

Ing. Jorge Arun Zambrano Cedeño, Mg.

Titulaciones	Semestre
• *Tecnologías de la Información en Línea*	*Quinto*

NOMBRES Y APELLIDOS: Mauro Vinicio Pomagualli Yambay.

PARALELO: "A"

SEMESTRE: Quinto

ACTIVIDAD: Sistema de Gestión de Incidentes (Help Desk)

PERIODO SEPTIEMBRE 2025 - ENERO 2026

Actividades a desarrollar en la Unidad 1

Actividad # 1: Estudio de Caso - Sistema de Gestión de Incidentes (Help Desk)

1. ANÁLISIS DEL PROBLEMA

1.1. Identificación de Actores Principales.

Los actores que interactúan con el sistema de Help Desk para la infraestructura del Data Center son:

- **Usuario Final:** Personal o cliente interno que opera o depende de los servidores y detecta anomalías en el servicio. Genera la solicitud inicial de asistencia.

- **Técnico de Soporte:** Especialista de TI encargado de recibir los incidentes asignados, diagnosticar la causa raíz física o lógica, ejecutar las acciones correctivas y actualizar el estado de resolución.

- **Administrador de Sistemas:** responsable global de la infraestructura de servidores y del sistema de Help Desk. Supervisa métricas de rendimiento, gestiona catálogos de infraestructura y recibe alertas reactivas críticas.

2. ESPECIFICACIÓN DE REQUERIMIENTOS

2.1. Requerimientos Funcionales.

- **Creación de Tickets:** El sistema debe permitir al Usuario Final registrar incidentes técnicos detallando la descripción de la falla del servidor.

- **Categorización Automática de Fallos:** El sistema debe clasificar el incidente obligatoriamente en tres áreas técnicas: Hardware (pide número de serie), Red (pide IP afectada) o Software (pide sistema operativo) mediante una clase de fabricación centralizada.

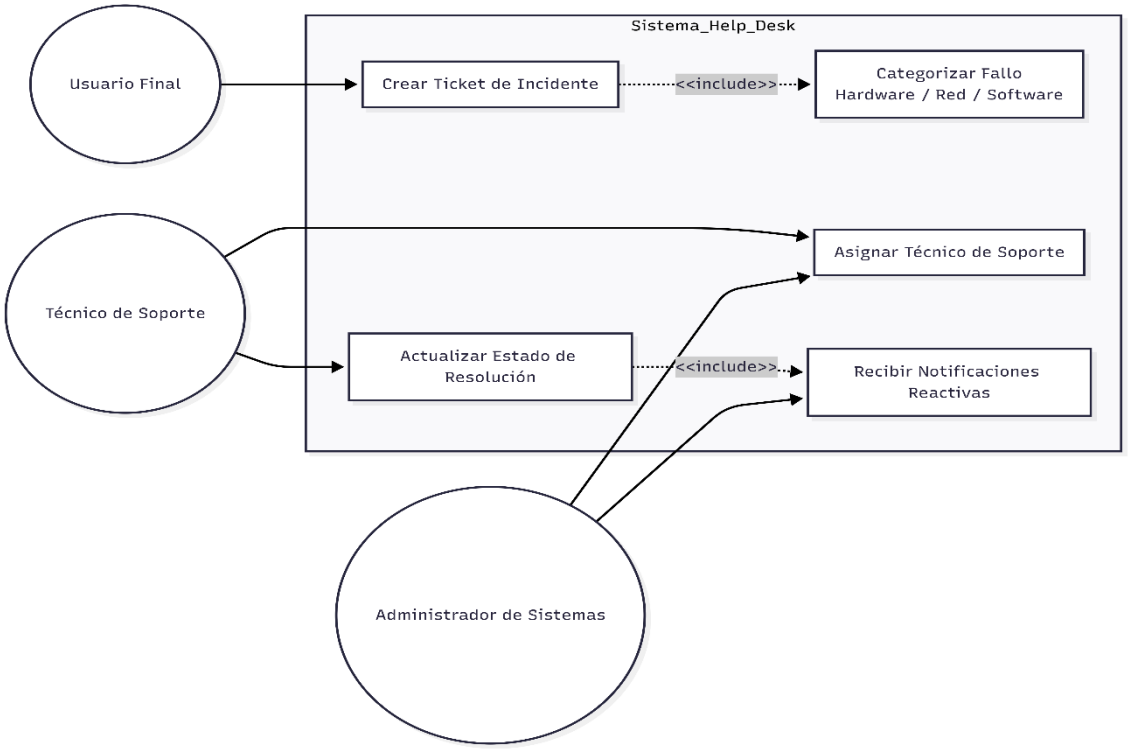
- **Asignación de Técnico:** El sistema debe permitir la vinculación de un Técnico de Soporte especializado al ticket según el área del fallo registrado.

- **Notificación Reactiva:** El sistema debe alertar de manera automatizada al Técnico de Soporte y al Administrador de Sistemas ante la aparición de nuevos incidentes o cambios de estado críticos.

2.2 Requerimientos No Funcionales.

- **Escalabilidad:** La arquitectura lógica debe soportar la adición de nuevos tipos de incidentes de infraestructura sin comprometer el rendimiento ni alterar los módulos existentes.

- **Mantenibilidad:** El código fuente debe estar estructurado bajo el patrón arquitectónico Model-View-Controller (MVC) para aislar la persistencia de datos de la lógica de negocio y presentación.
- **Trazabilidad de Versiones:** El ciclo de vida del desarrollo debe registrarse de manera estricta mediante un flujo de trabajo controlado para mitigar conflictos de código en entornos de producción.

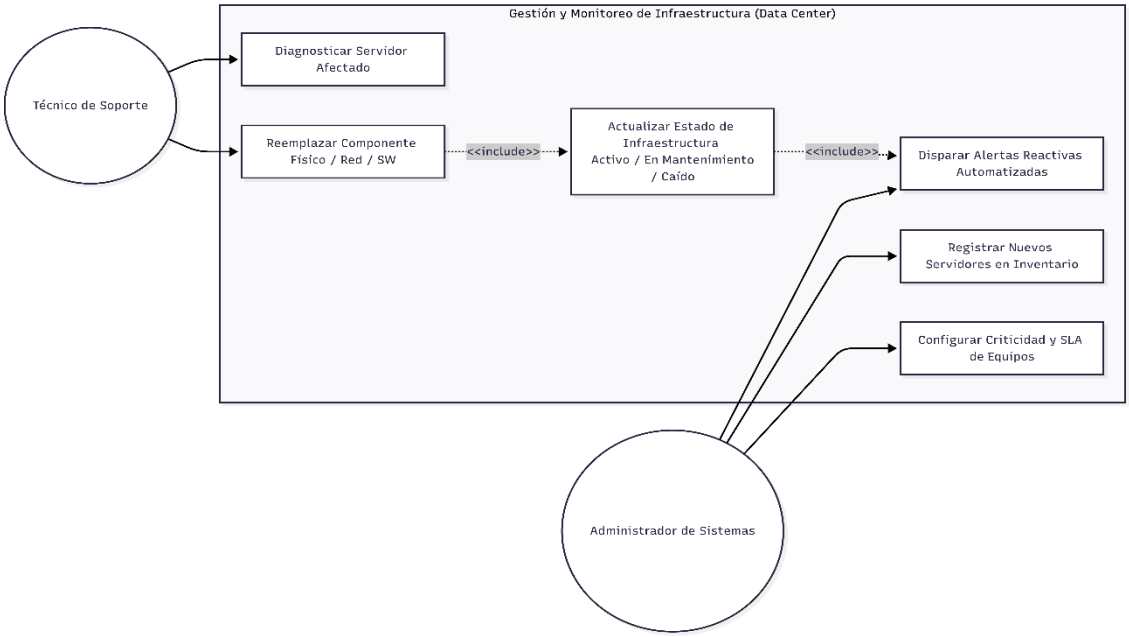


3. DISEÑO DEL SISTEMA

Para modelar específicamente cómo el personal técnico interactúa de manera directa con los componentes físicos y lógicos de la infraestructura del Data Center, necesitamos un diagrama enfocado en la administración operativa y no en el usuario común.

- **Técnico de Soporte:** Su interacción es operativa y reactiva sobre los activos. Diagnóstica fallas en tarjetas de red, módulos de memoria (Hardware), enrutamiento (Red) o sistemas operativos (Software).

- **Administrador de Sistemas:** Su interacción es preventiva, estructural y de monitoreo global. Administra el inventario de servidores, define políticas de criticidad y recibe las alertas del estado de la infraestructura.



4. ADIAGRAMA DE CLASES UML DONDE SE EVIDENCIEN LOS OBJETOS Y SU RELACIÓN BAJO PRINCIPIOS SOLID.

- **Single Responsibility Principle (SRP):** El MotorGestionTickets tiene la única responsabilidad de almacenar y centralizar los datos en memoria (Singleton). El GeneradorDeTickets solo sabe cómo instanciar objetos. El ControladorDeTicket solo maneja el flujo de peticiones. Ninguna clase interfiere en la tarea de otra, garantizando que cada componente tenga una única razón para cambiar (Martin, 2017).

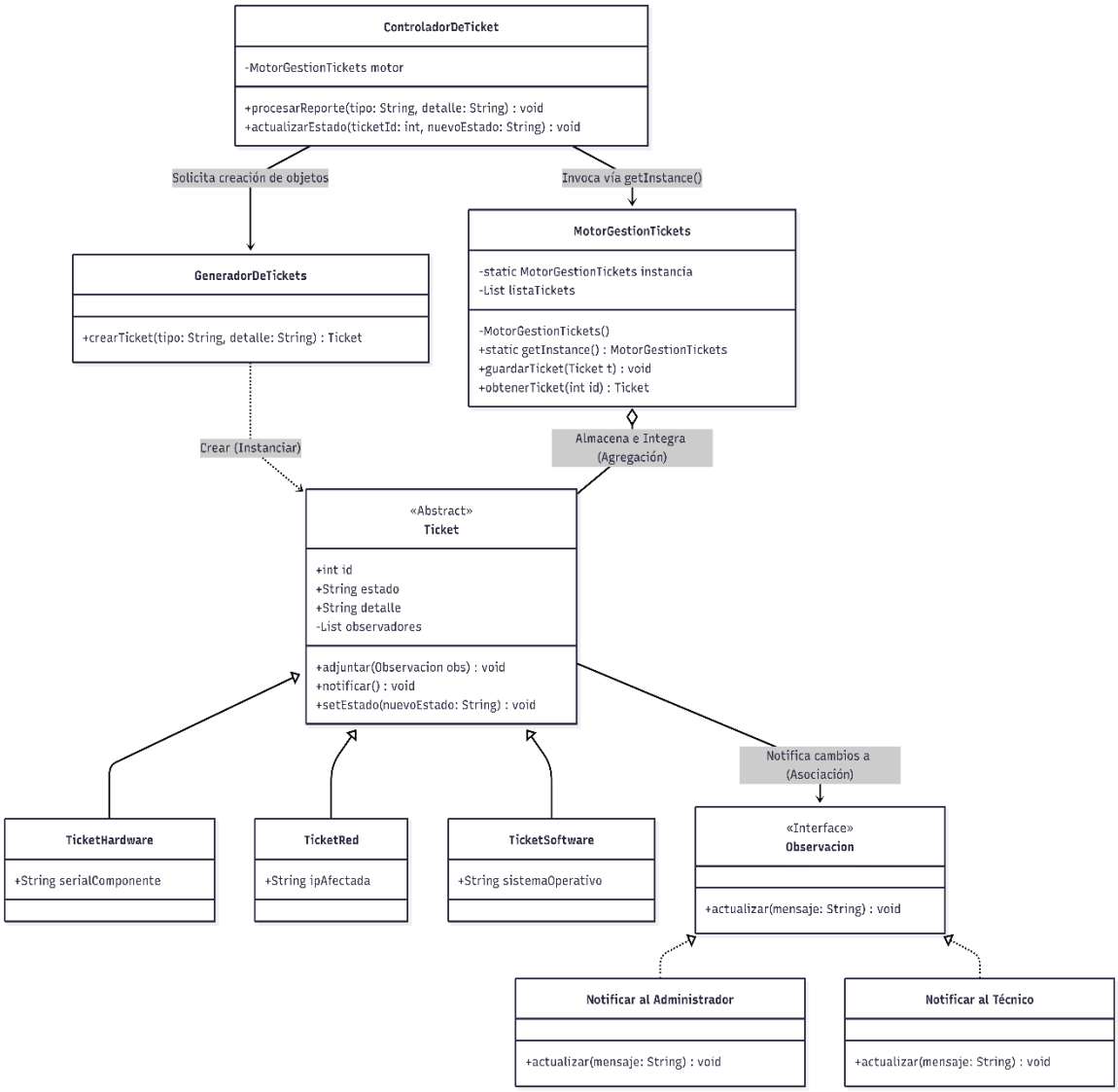
- **Open/Closed Principle (OCP):** El sistema está protegido ante cambios estructurales. Si el Data Center requiere añadir un nuevo tipo de infraestructura, como "Incidentes de Base de Datos", basta con crear la clase TicketDatabase heredando de Ticket y actualizar la lógica de la fábrica. Gracias a este principio, el resto del sistema —el controlador y el motor central— permanece intacto, permitiendo la extensión de funcionalidades sin riesgo de alterar el código fuente que ya es estable (Martin, 2017).

- **Liskov Substitution Principle (LSP):** Las clases derivadas (TicketHardware, TicketRed, TicketSoftware) extienden a la clase abstracta Ticket sin alterar su comportamiento base ni la integridad del sistema. Esto permite que el MotorGestionTickets almacene y procese cualquier variante de incidente dentro de su lista genérica de forma transparente, garantizando que los objetos de las subclases puedan sustituir a los objetos de la clase base sin afectar la exactitud del programa (Martin, 2017).

- **Interface Segregation Principle (ISP):** La interfaz Observacion es limpia y especializada, cumpliendo con la premisa de que ninguna clase debe ser forzada a depender de métodos que no utiliza. En el sistema de Help Desk, los notificadores solo implementan el método actualizar(), evitando así que se vean

obligados a heredar o conocer métodos de bajo nivel relacionados con la red o bases de datos que no corresponden a su función específica (Martin, 2017).

- **Dependency Inversion Principle (DIP):** La clase Ticket no conoce los detalles de implementación sobre cómo se envía una notificación, ya sea a través de correo electrónico, Slack o consola. Al depender directamente de la interfaz abstracta Observacion, el sistema logra desacoplar el núcleo del negocio de las herramientas externas de bajo nivel, asegurando que las políticas de alto nivel no dependan de los detalles, sino de abstracciones (Martin, 2017).



5. JUSTIFICACIÓN TÉCNICA DE LOS PATRONES UTILIZADOS.

a) Patrón Factory Method

- **Propósito:** centralizar e independizar la creación de objetos complejos o especializados a través de una interfaz común. En el sistema de Help Desk, permite que el Generador de Tickets instancie incidentes de Red, Hardware

o Software sin que el cliente necesite conocer las clases concretas, facilitando la extensión del sistema hacia nuevos tipos de infraestructura (Gamma et al., 1994).

- **Problema específico que resuelve en el Help Desk:** En un Data Center, un incidente de red requiere una dirección IP, uno de hardware pide un número de serie de la pieza y uno de software solicita el sistema operativo. Si la interfaz web creara estos objetos de forma directa, el código estaría lleno de condicionales difíciles de mantener. Este patrón resuelve el problema encapsulando esa lógica en el GeneradorDeTickets. Si en el futuro la infraestructura se amplía, se añade el nuevo tipo de ticket sin modificar el sistema existente.

b) Patrón Singleton

- **Propósito:** Garantizar que una clase tenga una única instancia en todo el ciclo de vida de la aplicación y proporcionar un punto de acceso global a ella. En el contexto del Help Desk, este patrón asegura que el MotorGestionTickets actúe como un núcleo centralizado y unificado, evitando la duplicidad de datos y garantizando que todos los componentes del sistema consulten la misma información de la infraestructura (Gamma et al., 1994).
- **Problema específico que resuelve en el Help Desk:** El motor de gestión “MotorGestionTickets” almacena la lista de incidentes y coordina el estado de la infraestructura en tiempo real. Si múltiples controladores crearan diferentes instancias del motor, los datos se duplicarían, se perdería la sincronización de los estados de los servidores y habría bloqueos en la persistencia de datos. El patrón Singleton resuelve esto asegurando que todos los componentes consulten exactamente la misma lista unificada de incidentes.

c) Patrón Observer

- **Propósito:** Definir una dependencia de uno a muchos entre objetos, de modo que cuando un objeto principal cambie su estado, todos sus dependientes sean notificados y actualizados de manera automática. En el sistema de Help Desk, esta arquitectura reactiva permite que cualquier cambio en el estado de un servidor o ticket dispare alertas inmediatas a los técnicos y administradores sin acoplar la lógica del incidente con los medios de comunicación (Gamma et al., 1994).

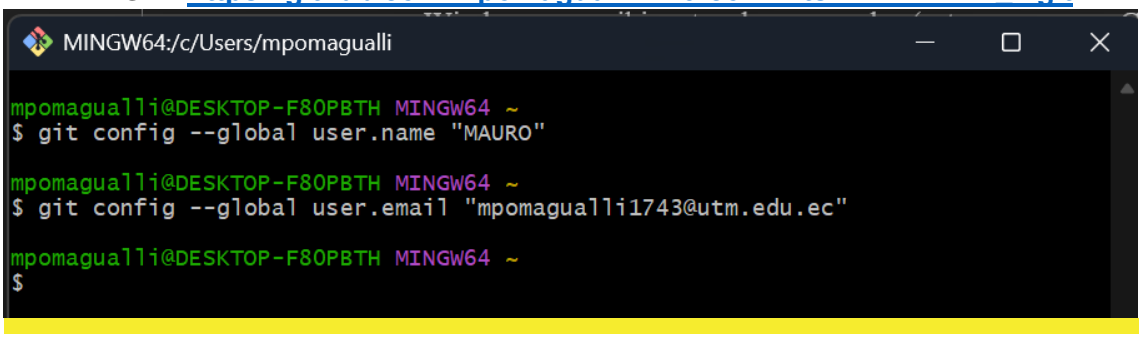
- **Problema específico que resuelve en el Help Desk:** Cuando un servidor crítico se cae o un técnico marca un ticket como "Resuelto", el sistema debe alertar de inmediato al Administrador de Sistemas y al Técnico asignado. Sin este patrón, el objeto Ticket tendría que conocer los detalles de cómo enviar correos, mensajes por Slack o SMS, generando un código acoplado y frágil. Con el patrón Observer (Base Reactiva), el ticket solo emite una señal de cambio; las clases de alerta NotificarAlAdministrador y NotificarAlTecnico reaccionan de manera independiente, permitiendo agregar nuevos canales de comunicación sin alterar la lógica.

d) Patrón Model-View-Controller - MVC

- **Propósito:** Dividir una aplicación en tres componentes interconectados para separar la representación interna de la información de las formas en que se presenta y se acepta la información del usuario. En el Help Desk, esta separación garantiza que la lógica de los servidores (Modelo) sea independiente de la interfaz de reporte (Vista), permitiendo que el controlador gestione las solicitudes de infraestructura de manera organizada y mantenible (Sommerville, 2011).
- **Problema específico que resuelve en el Help Desk:** Evita el anti patrón de "código espagueti", donde la interfaz de usuario (la consola o pantalla) maneja directamente las conexiones a la base de datos o las reglas de negocio. En el Help Desk, la vista solo captura el reporte; el ControladorDeTicket recibe la acción y coordina la lógica, y el Modelo (Ticket, GeneradorDeTickets) procesa la información de la infraestructura. Esto garantiza que, si hay cambios, la interfaz visual en el futuro, no se tendrá que reprogramar la lógica del Data Center.

6. EVIDENCIAS

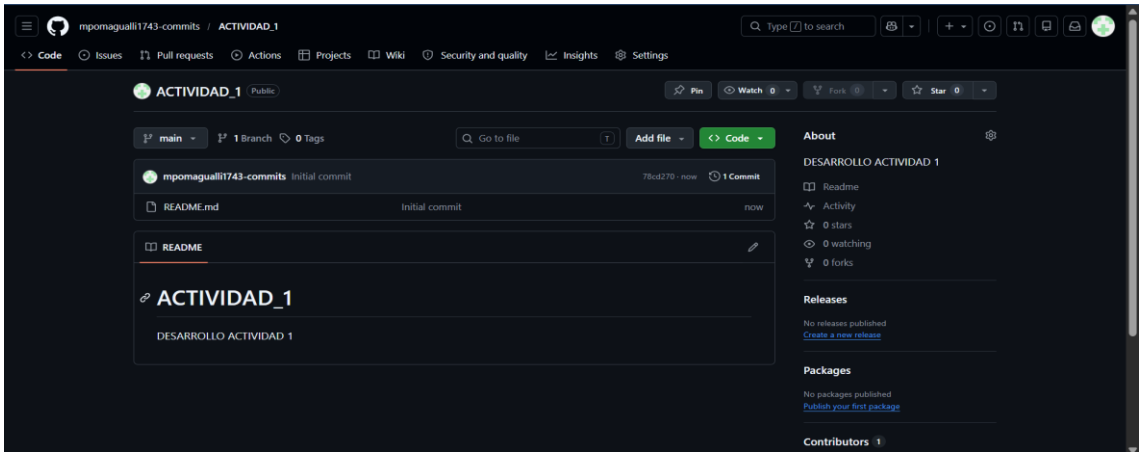
Link GIT: https://github.com/mpomagualli1743-commits/ACTIVIDAD_1.git



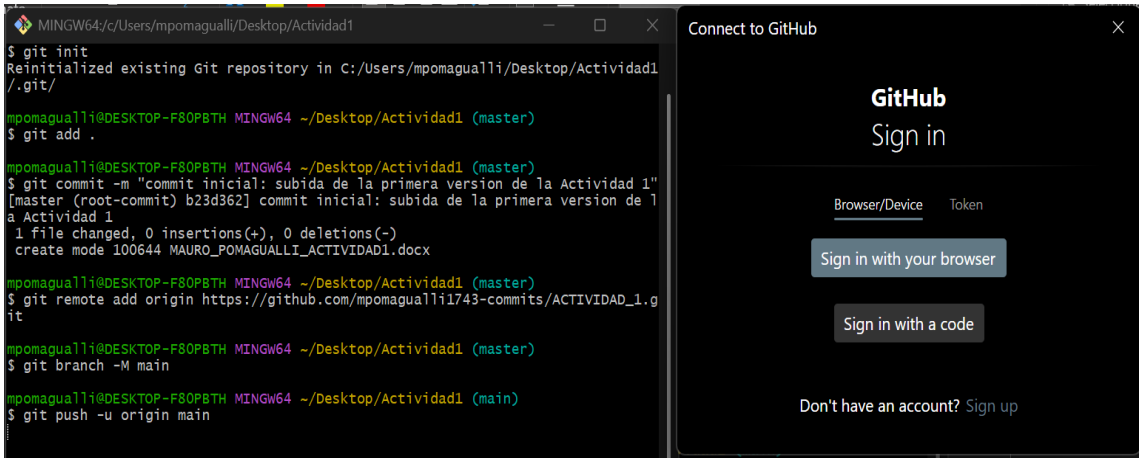
REALIZAMOS EL PRIMER COMMIT

```
MINGW64:/c/Users/mpomagualli/Desktop/Actividad1

mpomagualli@DESKTOP-F80PBTH MINGW64 ~/Desktop/Actividad1 (master)
$ git init
Reinitialized existing Git repository in C:/Users/mpomagualli/Desktop/Actividad1
/.git/

mpomagualli@DESKTOP-F80PBTH MINGW64 ~/Desktop/Actividad1 (master)
$ git add .

mpomagualli@DESKTOP-F80PBTH MINGW64 ~/Desktop/Actividad1 (master)
$ git commit -m "commit inicial: subida de la primera version de la Actividad 1"
[master (root-commit) b23d362] commit inicial: subida de la primera version de l
a Actividad 1
 1 file changed, 0 insertions(+), 0 deletions(-)
 create mode 100644 MAURO_POMAGUALLI_ACTIVIDAD1.docx

mpomagualli@DESKTOP-F80PBTH MINGW64 ~/Desktop/Actividad1 (master)
$
```

CREAMOS EL REPOSITORIO EN GITHUB



SE AÑADE UN DISPOSITIVO REMOTO, CRACIÓN DE UNA RAMA MAIN, Y REALIZAMOS EL PRIMER PUSH.



CICLO DE ACTUALIZACION

```
MINGW64:/c/Users/mpomagualli/Desktop/Actividad1
Everything up-to-date

mpomagualli@DESKTOP-F80PBTH MINGW64 ~/Desktop/Actividad1 (main)
$ git add .

mpomagualli@DESKTOP-F80PBTH MINGW64 ~/Desktop/Actividad1 (main)
$ git commit -m "actualizacion: se agrego evidencias de configuración de Git"
[main ffdbacc] actualizacion: se agrego evidencias de configuración de Git
 1 file changed, 0 insertions(+), 0 deletions(-)

mpomagualli@DESKTOP-F80PBTH MINGW64 ~/Desktop/Actividad1 (main)
$ git push -u origin main
Enumerating objects: 5, done.
Counting objects: 100% (5/5), done.
Delta compression using up to 8 threads
Compressing objects: 100% (3/3), done.
Writing objects: 100% (3/3), 297.88 KiB | 4.58 MiB/s, done.
Total 3 (delta 1), reused 0 (delta 0), pack-reused 0 (from 0)
remote: Resolving deltas: 100% (1/1), completed with 1 local object.
To https://github.com/mpomagualli1743-commits/ACTIVIDAD_1.git
   708d439..ffdbacc  main -> main
branch 'main' set up to track 'origin/main'.

mpomagualli@DESKTOP-F80PBTH MINGW64 ~/Desktop/Actividad1 (main)
$
```

Referencias

Martin, R. C. (2017). *Clean Architecture: A Craftsman's Guide to Software Structure and Design*. Prentice Hall.

Gamma, E., Helm, R., Johnson, R., y Vlissides, J. (1994). *Design patterns: Elements of reusable object-oriented software*. Addison-Wesley.

Sommerville, I. (2011). *Ingeniería de Software* (9na ed.). Pearson Educación.